Bioinformatics algorithms

Rosario M. Piro
Politecnico di Milano

Read mapping – exercises (**with solutions**)

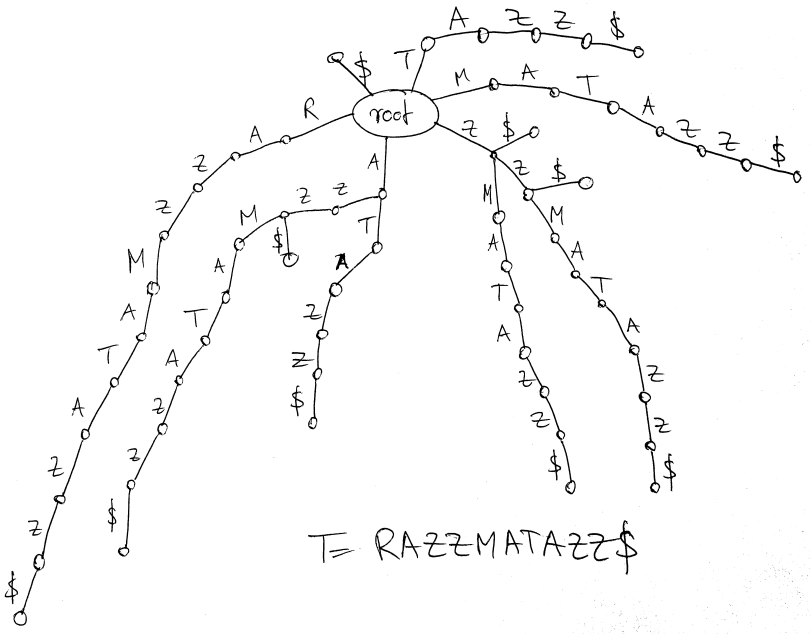
---

1. **Suffix trie and suffix tree**

   Given the word "RAZZMATAZZ":[1]

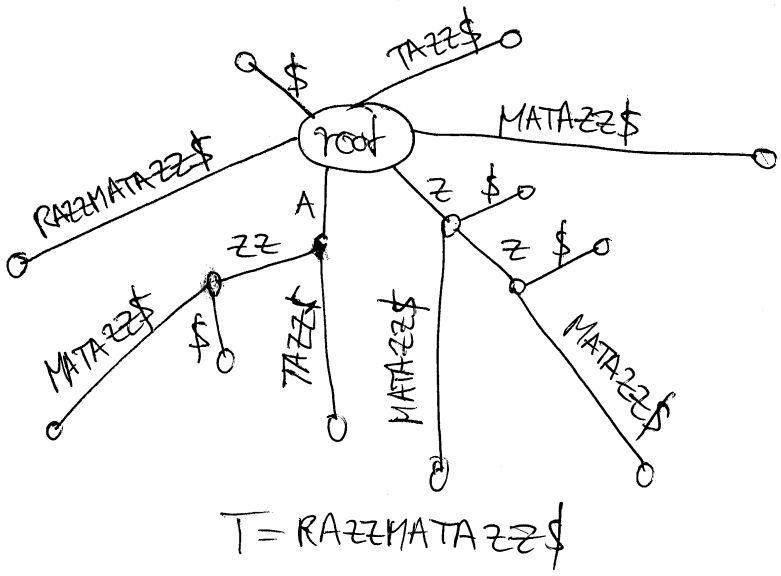
   (a) Build a suffix trie for the word.

   **Solution**




   (b) Compact the trie by combining non-branching paths.

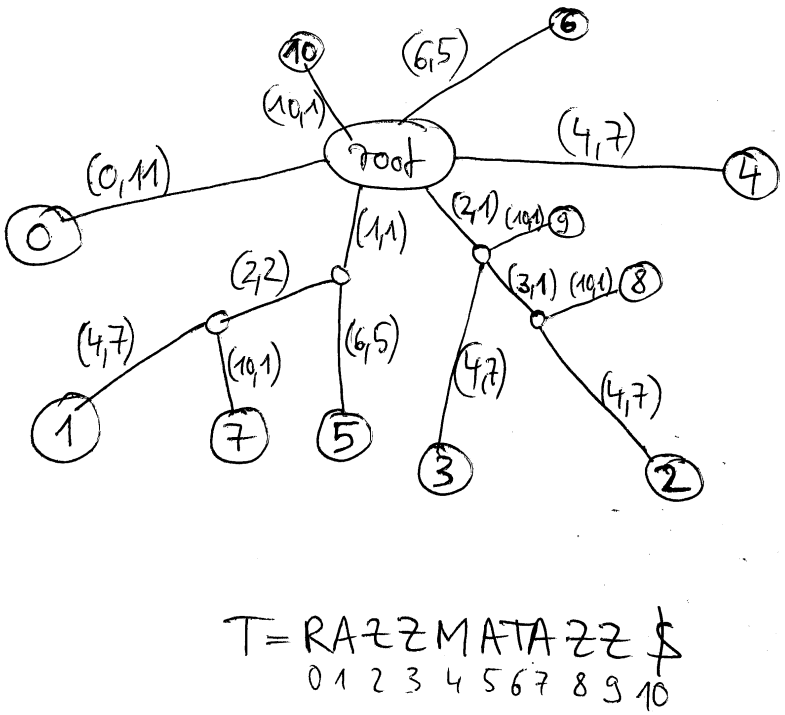
   **Solution**




[1] Meaning of "razzmatazz" from The Free Dictionary:
1. Excited action or impressive display.
2. Evasive or misleading language; double talk.

(c) Transform the compressed trie into a suffix tree.

**Solution**




(d) *Use the graphs* to answer the following questions:

1. How many times does the letter "A" occur?
2. What is the longest repeated substring?
3. At what positions in the original string does the longest repeated substring occur?

**Solution**

1. Three times "A" = three leaf nodes below prefix "A" (= three suffixes which start with "A")
2. "AZZ" of length 3 = lowest level with multiple leaf nodes
3. In the suffix tree: "A" corresponds to (1,1) and "ZZ" to (2,2); the leaf nodes below correspond to positions 1 and 7

(e) Remember the following algorithm to create a suffix trie (from the lectures):

**Algorithm 1** SuffixTrie($T$)

```
1: T+ = $
2: root = {}
3: for i=1 to i=length(T) do
4:    n = root{ # n is the current node}
5:    for c in T[i :] { # for each char in the i-th suffix} do
6:       if c ∉ n then
7:          n[c] = {} { # add outgoing edge to n if needed }
8:       end if
9:       n = n[c]{ # switch current node to child node }
10:   end for
11: end for
12: return root
```

How can you change the algorithm to build a prefix trie?

**Solution**
Best solution: don't actually change the algorithm. Instead reverse the input string and add a termination character as usual (which would correspond to a start character for the original string)!

In case we really want to change the algorithm to `PrefixTrie`($T$):
Line 1: `T = '^' + T`
Line 5: `for c in reverse(T[:i]) ...`
(We take prefixes from start to $i$ but process them backwards, so that the start character of the prefix will be at the leaf node and the end of the prefix at the root node of the trie!)

What if instead you want to build a suffix trie for the *reverse complement* of a nucleotide sequence?

**Solution**
Line 1: `T = '^' + T`
Line 5: `for c in reverse(T[:i]) ...`
Between lines 5 and 6: `replace` $c$ `A↔T, C↔G`
(Better solution: first determine the reverse complement (reverse + complement nucleotides), then use the original algorithm to build a suffix trie!)

2. **Suffix array**

(a) What is the suffix array (SA) for the word "RAZZMATAZZ"?

**Solution**

| | | | | |
|---|---|---|---|---|
| [0] | RAZZMATAZZ$ | | [10] | $ |
| [1] | AZZMATAZZ$ | | [5] | ATAZZ$ |
| [2] | ZZMATAZZ$ | | [7] | AZZ$ |
| [3] | ZMATAZZ$ | | [1] | AZZMATAZZ$ |
| [4] | MATAZZ$ | | [4] | MATAZZ$ |
| [5] | ATAZZ$ | $\Longrightarrow$ | [0] | RAZZMATAZZ$ |
| [6] | TAZZ$ | | [6] | TAZZ$ |
| [7] | AZZ$ | | [9] | Z$ |
| [8] | ZZ$ | | [3] | ZMATAZZ$ |
| [9] | Z$ | | [8] | ZZ$ |
| [10] | $ | | [2] | ZZMATAZZ$ |

Thus the suffix array is:

`SA = (10, 5, 7, 1, 4, 0, 6, 9, 3, 8, 2)`

(b) What is the selective SA for k=3 (every 3rd)?

**Solution**

`SA = (-, -, -, -, -, 0, 6, 9, 3, -, -)`

Important: we keep every 3rd position in the *original string*, not every third element in the suffix array!

(c) Using the information from the suffix array, answer the following questions:

1. At what positions does the letter "Z" occur in the original string?
2. How can you use this information to identify the longest repeated substring?

**Solution**

1. From the last four entries of the SA: positions 9, 3, 8 and 2
2. Substrings are prefixes of suffixes, thus we look for the longest common prefix of consecutive SA entries: "AZZ" in entries 3 and 4 (corresponding to positions 7 and 1 in the original string)

(d) (Optional; for fun; no solution here) Implement the Manber-Myers algorithm to sort a suffix array!

3. **Burrows-Wheeler transform**

(a) Compute the Burrows-Wheeler transform (BWT) for the input string "ACGTTGAACTT" (without the quotation marks)

- ... by performing rotations
- ... by first computing the suffix array and from this the BWT

**Solution**

Easiest approach (rotations):

1. Don't forget to add the termination character ($)!
2. Get all string rotations
3. Sort the rotated strings
4. Extract all last characters (L-column) to build the BWT

| | | |
|---|---|---|
| ACGTTGAACTT$ | | $ACGTTGAACTT |
| CGTTGAACTT$A | | AACTT$ACGTTG |
| GTTGAACTT$AC | | ACGTTGAACTT$ |
| TTGAACTT$ACG | | ACTT$ACGTTGA |
| TGAACTT$ACGT | | CGTTGAACTT$A |
| GAACTT$ACGTT | $\Longrightarrow$ | CTT$ACGTTGAA |
| AACTT$ACGTTG | | GAACTT$ACGTT |
| ACTT$ACGTTGA | | GTTGAACTT$AC |
| CTT$ACGTTGAA | | T$ACGTTGAACT |
| TT$ACGTTGAAC | | TGAACTT$ACGT |
| T$ACGTTGAACT | | TT$ACGTTGAAC |
| $ACGTTGAACTT | | TTGAACTT$ACG |

Therefore, the BWT is "TG$AAATCTTCG" (last column)

Alternative approach (SA):

1. Don't forget to add the termination character ($)!
2. Construct the suffix array (SA):

| | | | | |
|---|---|---|---|---|
| [0] | ACGTTGAACTT$ | | [11] | $ |
| [1] | CGTTGAACTT$ | | [6] | AACTT$ |
| [2] | GTTGAACTT$ | | [0] | ACGTTGAACTT$ |
| [3] | TTGAACTT$ | | [7] | ACTT$ |
| [4] | TGAACTT$ | | [1] | CGTTGAACTT$ |
| [5] | GAACTT$ | $\Longrightarrow$ | [8] | CTT$ |
| [6] | AACTT$ | | [5] | GAACTT$ |
| [7] | ACTT$ | | [2] | GTTGAACTT$ |
| [8] | CTT$ | | [10] | T$ |
| [9] | TT$ | | [4] | TGAACTT$ |
| [10] | T$ | | [9] | TT$ |
| [11] | $ | | [3] | TTGAACTT$ |

$\Longrightarrow SA = (11, 6, 0, 7, 1, 8, 5, 2, 10, 4, 9, 3)$

3. Convert the SA into the BWT:

- $BWT[i] = T[SA[i]-1]$ for $SA[i] > 0$ and
- $BWT[i] = \$$ for $SA[i] = 0$

with $T$:

```
ACGTTGAACTT$
          11
012345678901
```

Conversion:

| $i$ | $SA[i]$ | $SA[i]-1$ | $T[SA[i]-1]$ | $BWT[i]$ |
|---|---|---|---|---|
| 0 | 11 | 10 | T | T |
| 1 | 6 | 5 | G | G |
| 2 | 0 | – | – | $ |
| 3 | 7 | 6 | A | A |
| 4 | 1 | 0 | A | A |
| 5 | 8 | 7 | A | A |
| 6 | 5 | 4 | T | T |
| 7 | 2 | 1 | C | C |
| 8 | 10 | 9 | T | T |
| 9 | 4 | 3 | T | T |
| 10 | 9 | 8 | C | C |
| 11 | 3 | 2 | G | G |

(b) (Optional; for fun; no solution here) Write a program that takes an input string and returns it's BWT. No need for efficiency. Test your program with the input string from above! If you want a more challenging task: write a program which first computes the suffix array using the Manber-Myers sorting and then converts the SA into the BWT.

4. **Burrows-Wheeler transform 2**

(a) Given the $BWT$="CGGTCGAATGA$ATTTTA" (without the quotation marks), what is the original string? First use a "graphical approach" ...

**Solution**

| $i$ | $F[i]$ | | $L[i]$ |
|---|---|---|---|
| 0 | $\$_0$ | $\rightarrow$ | $C_0$ |
| 1 | $A_0$ | | $G_0$ |
| 2 | $A_1$ | | $G_1$ |
| 3 | $A_2$ | | $T_0$ |
| 4 | $A_3$ | | $C_1$ |
| 5 | $A_4$ | | $G_2$ |
| 6 | $C_0$ | | $A_0$ |
| 7 | $C_1$ | | $A_1$ |
| 8 | $G_0$ | | $T_1$ |
| 9 | $G_1$ | | $G_3$ |
| 10 | $G_2$ | | $A_2$ |
| 11 | $G_3$ | | $\$_0$ |
| 12 | $T_0$ | | $A_3$ |
| 13 | $T_1$ | | $T_2$ |
| 14 | $T_2$ | | $T_3$ |
| 15 | $T_3$ | | $T_4$ |
| 16 | $T_4$ | | $T_5$ |
| 17 | $T_5$ | | $A_4$ |

We reconstruct the original string $T$ starting in reverse order, i.e., starting from the termination character $\$_0$ (in the F-column):

- The preceding character (L-column) is $C_0$ (i.e., the C with B-rank 0)!
- We find $C_0$ at $i = 6$ in the F-column; preceding character is $A_0$!
- We find $A_0$ at $i = 1$ in the F-column; preceding character is $G_0$!
- ...

Thus, the original string is $T$ = $\texttt{G}_3\texttt{G}_1\texttt{A}_1\texttt{C}_1\texttt{A}_3\texttt{T}_0\texttt{A}_2\texttt{G}_2\texttt{A}_4\texttt{T}_5\texttt{T}_4\texttt{T}_3\texttt{T}_2\texttt{T}_1\texttt{G}_0\texttt{A}_0\texttt{C}_0\$_0$ = GGACATAGATTTTTGAC$

(b) For the BWT above ($BWT$ = CGGTCGAATGA$ATTTTA), define the following additional data structures:

1. For the F-column: the vector $C(a)$ of cumulative counts for all symbols $a \in \Sigma$ (here: all nucleotides)
2. An individual vector $Occ(i) = Occ(BWT[i], i)$ counting the number of occurrences of character $a = BWT[i]$ until position $i$ in the BWT, where $BWT[i]$ is precisely the character at that position (e.g., if $BWT[i]$ is "T", then $Occ(i) = Occ(BWT[i], i)$ is the number of "T"s in $BWT$[0:i])
   Note: this is similar to the tally table, but the information is summarized in one vector!

**Solution**

| $a \in \Sigma$ | $ | A | C | G | T |
|---|---|---|---|---|---|
| $n_a$ | 1 | 5 | 2 | 4 | 6 |
| $C(a)$ | 0 | 1 | 6 | 8 | 12 |

| $i$ | $F[i]$ | $BWT[i]$ | $Occ(i) = Occ(BWT[i], i)$ |
|---|---|---|---|
| 0 | $ | C | 1 |
| 1 | A | G | 1 |
| 2 | A | G | 2 |
| 3 | A | T | 1 |
| 4 | A | C | 2 |
| 5 | A | G | 3 |
| 6 | C | A | 1 |
| 7 | C | A | 2 |
| 8 | G | T | 2 |
| 9 | G | G | 4 |
| 10 | G | A | 3 |
| 11 | G | $ | 1 |
| 12 | T | A | 4 |
| 13 | T | T | 3 |
| 14 | T | T | 4 |
| 15 | T | T | 5 |
| 16 | T | T | 6 |
| 17 | T | A | 5 |

(c) Now write a short algorithm (pseudo code) which uses these two additional data structures, along with the BWT, to reconstruct the original string $T$!

**Solution**

**Algorithm 2** ReverseBurrowsWheeler(BWT)

```
1: T = BWT[0] + '$'                    # last character is BWT[0] since F[0] is '$'!
2: i = 0                               # current index; start in first row!
3: for n=1 to n=length(BWT)-2 do
4:   i = C[BWT[i]] + Occ[i] - 1        # update index in F
5:   T = BWT[i] + T                    # add next character from BWT at new index i
6: end for
7: return T
```

Applying this: initializing for $i = 0$ we start with $T$="C$"

1. For $n = 1$ ($i$=0; $BWT$[0]='C'): $i$ = 6 + 1 - 1 = 6; $T$="AC$"
2. For $n = 2$ ($i$=6; $BWT$[6]='A'): $i$ = 1 + 1 - 1 = 1; $T$="GAC$"
3. For $n = 3$ ($i$=1; $BWT$[1]='G'): $i$ = 8 + 1 - 1 = 8; $T$="TGAC$"
4. For $n = 4$ ($i$=8; $BWT$[8]='T'): $i$ = 12 + 2 - 1 = 13; $T$="TTGAC$"
5. ...

6. For $n = 15$ ($i$=2; $BWT$[2]='G'): $i$ `= 8 + 2 - 1 = 9`; $T$="GACATAGATTTTTGAC$"
7. For $n = 16$ ($i$=9; $BWT$[9]='G'): $i$ `= 8 + 4 - 1 = 11`; $T$="GGACATAGATTTTTGAC$"

(d) In the lecture we have seen, that for searching occurrences of substrings (e.g., reads in the genome), we would use the FM index, composed of the BWT, a representation of the F-column, a (selective) suffix array and the tally table (checkpoints). Using instead only the structures we have so far defined in this exercise ($BWT$, $C(a)$ and $Occ(BWT[i], i)$), would it be possible to

1. ... determine the number of occurrences of a substring?
2. ... identify their positions within the original string $T$?

If so, how? (A conecptual description is enough, no precise algorithm needs to be developed)

**Solution**

1. Yes! In each step, we would need to find a range of indices which represent the partial solution (i.e., matches of the partial substring, determined backwards). This requires to know the B-ranks of letters we identify in same range of the BWT. The B-ranks would usually be determined from the tally table, but they can actually also be determined from $Occ(i) = Occ(BWT[i], i)$ by using a similar approach we use in case we only selectively saved certain entries: we walk back until we find, what we need!
   - Example: let's say we search for a substring which ends in "...AG"
   - First, we find all "G"s in the F-column and get an initial index range of [8,11] for the partial solution "G"
   - Then, we have to look for the B-ranks of all "A"s in $BWT$[8:11], so that we can find the corresponding "A"s in the F-column to get the new index range for the partial solution "AG"
   - How can we get the B-ranks of the "A"s in in $BWT$[8:11]? We walk back from $BWT$[11] until we encounter an "A" (here: at index 10) and check it's B-rank in $Occ(i)$ (here: 3). So the last "A" we need is "$A_3$"
   - We then walk forward from $BWT$[8] until we encounter an "A" (here: also at index 10) and check it's B-rank in $Occ(i)$ (here: 3). So the first "A" we need is "$A_3$"
   - Thus, in this case, for the partial solution "AG", we need to look only in the F-column which contains "$A_3$" (only this index corresponds to an "A" which is followed by "G"!)
   - Had we instead not found any "A" between $BWT$[8] and $BWT$[11], then there would be no solution and hence no occurrence of "...AG" in the original string!
   - (Additional note: if a single vector $Occ$ is potentially enough, then why do we use the tally table in the FM index? Because the search in the range $[a, b]$ of the vector $Occ$ has theoretically a time complexity of $O(|T|)$; so it depends of the length of the reference genome ...)
2. No! For this purpose we need the SA because only this data structure contains information about the original positions of the suffixes

5. **FM index**

(a) For the following input string, build the full FM index with k=3 for the selective SA and x=4 for the tally table checkpoints:

"razzledazzle" (without quotation marks)

**Solution**

$T$ = `razzledazzle$`

- BWT: `edrellzz$zzaa`

- F-column:

| $a \in \Sigma$ | $ | a | d | e | l | r | z |
|---|---|---|---|---|---|---|---|
| # occurrences | 1 | 2 | 1 | 2 | 2 | 1 | 4 |
| $C(a)$ = total # below $a$ | – | 0 | 2 | 3 | 5 | 7 | 8 |

- SA: (12, 7, 1, 6, 11, 5, 10, 4, 0, 9, 3, 8, 2) $\Longrightarrow$ selective SA (k=3): (12, –, –, 6, –, –, –, –, 0, 9, 3, –, –)

- tally table:

| $i$ | $BWT[i]$ | a | d | e | l | r | z |
|---|---|---|---|---|---|---|---|
| **0** | e | **0** | **0** | **1** | **0** | **0** | **0** |
| 1 | d | 0 | 1 | 1 | 0 | 0 | 0 |
| 2 | r | 0 | 1 | 1 | 0 | 1 | 0 |
| 3 | e | 0 | 1 | 2 | 0 | 1 | 0 |
| **4** | l | **0** | **1** | **2** | **1** | **1** | **0** |
| 5 | l | 0 | 1 | 2 | 2 | 1 | 0 |
| 6 | z | 0 | 1 | 2 | 2 | 1 | 1 |
| 7 | z | 0 | 1 | 2 | 2 | 1 | 2 |
| **8** | $ | **0** | **1** | **2** | **2** | **1** | **2** |
| 9 | z | 0 | 1 | 2 | 2 | 1 | 3 |
| 10 | z | 0 | 1 | 2 | 2 | 1 | 4 |
| 11 | a | 1 | 1 | 2 | 2 | 1 | 4 |
| **12** | a | **2** | **1** | **2** | **2** | **1** | **4** |

(The selective checkpoints we keep for x=4 are indicated in **bold-face** font)

(b) Use the data structures from the FM index to search for the occurrences and positions of the substring "azzl" ... Here, report the evolving index ranges for the single steps that you obtain for the partial solutions and the final positions within the original string

**Solution**

- Step 1 ("l"): index range in F is [6:7]
- Step 2 ("zl"): index range in F is [9:10]
- Step 3 ("zzl"): index range in F is [11:12]
- Step 4 ("azzl"): index range in F is [1:2]
- Thus, we have two occurrences (rows at indices 1 and 2 of the F-column) at positions 7 and 1 (obtained from the SA for indices 1 and 2)

6. **Inexact matching**

Given the original string $T$="razzledazzle$" and the alphabet $\Sigma$=(a,d,e,l,r,z):

(a) Precalculate the lower bounds $D(i)$ of mismatches for the search word $W$="hazle".

**Solution** Using the algorithm `CalculateD(W)` from the lecture:

- Step 1: j=0, i=0 ($\rightarrow W[0..0]$): "h" not in $T \rightarrow$ D(0) = 1 (increase of minimum mismatches)
- Step 2: j=1, i=1 ($\rightarrow W[1..1]$): "a" in $T \rightarrow$ D(1) = 1
- Step 3: j=1, i=2 ($\rightarrow W[1..2]$): "az" in $T \rightarrow$ D(2) = 1
- Step 4: j=1, i=3 ($\rightarrow W[1..3]$): "azl" not in $T \rightarrow$ D(3) = 2 (increase of minimum mismatches)
- Step 5: j=4, i=4 ($\rightarrow W[4..4]$): "e" in $T \rightarrow$ D(4) = 2

So $D = (1, 1, 1, 2, 2)$

(b) Consider the algorithm `InexRecur(W,i,z,k,l)` used to recursively find possible matches allowing for up to $d_{max}$ errors. Explain how you can use $D(i)$, computed above, to early interrupt unsuccessful mapping attempts for searching $W$="hazle" with $d_{max} = 1$.

**Solution**

- Iteration 1: Start with `InexRecur`($W$, $|W|-1$, $d_{max}$, 1, $|T|-1$) = `InexRecur(W,4,1,1,12)`, i.e., checking for the last letter of the search word $W(4)$
  - Lines 1-3: `i`=4 is not less than 0, so we haven't finished
  - Lines 4-6: Within `InexRecur` we note that `z`=$d_{max}$ < $D(4)$=2, so there is no solution! The word "hazle" will have at least 2 mismatches!

We can immediately interrupt the search without comparing a single letter ...

(c) Given $D(i)$, computed above, execute a full search of $W$="hazle" with $d_{max} = 2$. No need to compute all SA intervals (lines 9-10 of the algorithm), just explain what happens in the individual iterations/steps!

**Solution**

- Iteration 1: Start with `InexRecur`($W$, $|W|-1$, $d_{max}$, 1, $|T|-1$) = `InexRecur(W,4,2,1,12)`, i.e., checking for the last letter of the search word $W(4)$
  - Lines 1-3: `i`=4 is not less than 0, so we haven't finished
  - Lines 4-6: `z`=$d_{max}$ is not less than $D(4) = 2$, so we can try this iteration
  - Lines 7-19: try to match each letter of the alphabet $b_1 \in \Sigma$ and compute the corresponding new SA interval $[k,l]_{b_1}$. If $b_1$==$W(4)$, call `InexRecur(W,3,2,k,l)`, allowing still 2 mismatches; otherwise we have a first mismatch and call `InexRecur(W,3,1,k,l)`, allowing for the rest of the search word only one further mismatch!

- Iteration 2A ($b_1 \neq$ '$e$'; mismatch in iteration 1): `InexRecur(W,3,1,k,l)`
  - Lines 1-3: `i`=3 is not less than 0, so we haven't finished
  - Lines 4-6: `z`=1 < $D(3)$=2, so we can interrupt this attempt. We allow only 1 further mismatch but will have at least 2 for "hazl"! We return NULL (for $d_{max} = 2$, the original string $T$ has no approximately matching substrings which end with a character different from '$e$' because these would have at least 3 mismatches!)
- Iteration 2B ($b_1 =$ '$e$'; match in iteration 1): `InexRecur(W,3,2,k,l)`
  - Lines 1-3: `i`=3 is not less than 0, so we haven't finished
  - Lines 4-6: `z`=2 is not less than $D(3) = 2$, because we hadn't had a mismatch so far, continue
  - Lines 7-19: try to match each letter of the alphabet $b_2 \in \Sigma$ and compute the corresponding new SA interval $[k,l]_{b_2}$. If $b_2$==$W(3)$, call `InexRecur(W,2,2,k,l)`; otherwise we have a mismatch and would usually call `InexRecur(W,2,1,k,l)`, allowing for the rest of the search word only one further mismatch, but:
    * In this case, we will not get any valid SA interval for $b_2 \neq W(3)$ (we get $k>l$ in line 11!) because the original string $T$ doesn't contain any substring "$<b_2>e$" with $b_2 \neq$ '$l$', so we do not call `InexRecur(W,2,1,k,l)` and skip to the next character from the alphabet ...
    * Line 11 implements an important intuition: whenever adding a further letter $b \in \Sigma$ to the current match would lead to a (partial) word which does not exist in the original string $T$, we will get an invalid SA interval (because there is no suffix in $T$ with starts with this word)! We need to compare only words which are present in $T$ to the search pattern $W$ (to check how many mismatches they have)! Or, in our biological context: if we want to align a given read ($W$) to the reference genome ($T$), then we of course need to compare that read only to DNA sequences which actually exist in the reference genome!

- Iteration 3A ($b_2 =$ '$l$'; match in iteration 2B): `InexRecur(W,2,2,k,l)`
  - Lines 1-3: `i`=2 is not less than 0, so we haven't finished
  - Lines 4-6: `z`=2 is not less than $D(2) = 1$, continue
  - Lines 7-19: try to match each letter of the alphabet $b_3 \in \Sigma$ and compute the corresponding new SA interval $[k,l]_{b_3}$. If $b_3$==$W(2)$, call `InexRecur(W,1,2,k,l)`, otherwise we have a mismatch and would call `InexRecur(W,1,1,k,l)`, allowing for the rest of the search word only one further mismatch!

    * Actually, also here we will not get any valid SA interval for $b_3 \neq W(2)$ because the original string $T$ doesn't contain any substring "$<b_3>le$" with $b_3 \neq$ '$z$', so we won't even call `InexRecur(W,1,1,k,l)` ...

- Iteration 4A ($b_2 =$ '$z$'; match in iteration 3A): `InexRecur(W,1,2,k,l)`
  - Lines 1-3: `i`=1 is not less than 0, so we haven't finished
  - Lines 4-6: `z`=2 is not less than $D(1) = 1$, continue
  - Lines 7-19: try to match each letter of the alphabet $b_4 \in \Sigma$ and compute the corresponding new SA interval $[k, l]_{b_4}$. If $b_4 == W(1)$, we would call `InexRecur(W,0,2,k,l)`, otherwise we have a mismatch and call `InexRecur(W,0,1,k,l)`, allowing for the rest of the search word only one further mismatch!
    * Actually, in this case we will not get any valid SA interval for $b_4 = W(1)$ because the original string $T$ doesn't contain any substring "$<b_4>zle$" with $b_4 =$ '$a$', so we won't call `InexRecur(W,0,2,k,l)` ...
    * The same consideration holds for most other letters; we have only "$zzle$" in $T$, so only for $b_4 =$ '$z$' we get a valid SA interval and descend into `InexRecur(W,0,1,k,l)`!

- Iteration 5A ($b_4 \neq$ '$a$', indeed $b_4 =$ '$z$'; mismatch in iteration 4A): `InexRecur(W,0,1,k,l)`
  - Lines 1-3: `i`=0 is not less than 0, so we haven't finished!
  - Lines 4-6: `z`=1 is not less than $D(0)$=1, continue
  - Lines 7-19: try to match each letter of the alphabet $b_5 \in \Sigma$ and compute the corresponding new SA interval $[k, l]_{b_5}$. Since $W(0)$='$h$' $\notin \Sigma$, all possible $b_5 \in \Sigma$ constitute a mismatch and we call `InexRecur(W,-1,0,k,l)`
    * Actually, we will get a valid SA interval only for $b_5 =$ '$a$' because $T$ has only occurrences of '$a$' before "$zzle$" ...

- Iteration 6A ($b_5 \neq$ '$h$', indeed $b_5 =$ '$a$'; mismatch in iteration 5A): `InexRecur(W,-1,0,k,l)`
  - Lines 1-3: `i`=-1 is less than 0, we have finished! Just return the SA interval $[k, l]$ we determined from the previous iteration (here: $[k, l]_{b_5}$ from iteration 5A)!

We will find two occurrences of "$hazle$" with $d_{max} = 2$ at the following 0-based positions in the original string $T$ (these positions are those contained in the SA at the identified SA interval):

1. "azzle" at offset/position 1
2. "azzle" at offset/position 7

Note: in this case, since both matching substrings have the same sequence, we obtained only one SA interval, but if different words in $T$ match the search pattern with the given maximum mismatches, then we can also get multiple SA intervals!

(Optional: try to execute the algorithm with the iterative computation of the SA intervals and see if you actually get the positions 1 and 7 of the original string $T$ as a final result ...)